Foundation

Inventory States and Allocation Design

Written in July-2025 by Manufacturing Cloud (MFG-Vikings team) to describe the conceptual design of Inventory States and Allocation design for engineers and architects across Industries Cloud.

Version History

Version Number	Summary	Author
1	Abstract, Design goals	Naveen Kumar
2	Data model, architecture diagram, approaches	Naveen Kumar
3	UI layer	Muskan Agarwal
4	Allocation APIs	Chitra R
5	Auto allocation	Shubham Kishore

Reviewers

Reviewer	Status	Role	Date	Comments
Kusuma Kumari Seshavarapu	✅⚠️❌	Architect		
Naveen Kumar Prasad Veluri Manju Prasad	✅⚠️❌	Lead		
Deepak Jakkampudi Mohammed Bilal Sattikar	✅⚠️❌	Product Manager		
Vaibhav Jain Praveen Jain	✅⚠️❌	Engineering Manager		

Abstract

The introduction of inventory statuses and allocation capabilities is a foundational step toward building a robust inventory management system within the Salesforce platform. In the previous release, we had added the support for various inventory statuses such as allocation, ordered, in-transit, damaged thus providing a more granular view of inventory state.

This document outlines the design of the Inventory Allocation framework which includes user interfaces to allocate inventory, APIs to handle inventory allocations and the processing layer that takes care of synchronising allocation requests for various locations and products with the actual inventory records.

Business Drivers / Problem Statement

Enable inventory managers to allocate stock for specific orders and distinguish between available inventory and soft-reserved inventory for existing commitments. This capability prevents over-promising, improves order fulfillment accuracy, and ensures clear visibility into operational health.

For example:

- In **Manufacturing**, managing service parts for repairs and track finished goods as they move from the factory to distributors.
- In **Automotive**, serialized parts and vehicles must be tracked with precision, and statuses can indicate if a specific vehicle is in the showroom, allocated to a customer order, or undergoing pre-delivery inspection.
- In a **DMS context**, distributors need to manage their own operations effectively, including procurement from OEMs and sales to retailers. Detailed DMS use case diagram below.

PRD: MFG 258 Epic - DMS , Allocation epic document **Product Brief:** Link **Product vision:** Link

Team

Sponsoring cloud	Manufacturing Cloud
Engineering responsible	Naveen Kumar Prasad Veluri
PM responsible	Deepak Jakkampudi Mohammed Bilal Sattikar
Q3/Q4 responsible	Sindhuri Grandhi Satish Kumar Nagalla
Sign off stakeholders	
Architect responsible	Kusuma Kumari Seshavarapu

Design Goals

The goal is to introduce a mechanism for soft reserving inventory for various types of demand (sales orders,work orders etc), ensuring accurate tracking of available, on-hand, and allocated stock. This new functionality will be built around a dedicated inventory allocation entity, supported by flexible APIs and designed to handle high-volume and concurrent transactions. Here are few guiding principles -

- **Decoupling:** The core allocation logic should be decoupled from the specific systems that consume it (e.g., Sales Orders, Work Orders) to ensure flexibility and reusability.
- **Scalability:** The system must be designed to handle high volumes of orders and concurrent allocation requests, a key requirement for customers like the CG team.
- **Flexibility:** The data model and API design should be flexible enough to accommodate various order types and potential future enhancements without requiring major rework.
- Should support standard products in 260. Serialized products and batch will be 260+

Out of Scope

- **Complex Reallocation Logic:** Advanced logic for prioritizing orders and automatically reallocating inventory from low-priority to high-priority orders is not in the scope of the initial release.
- **Order states :** The allocation step is not coupled with any order state. Order allocation and deallocation should be handled by customers as per their requirement. We expose APIs which can be consumed in their work flows or directly use our UI.
- **Advanced UI:** Detailed UI mockups and implementation are deferred to a later phase.
- **Intelligent Location Selection:** The system will not automatically determine the optimal location to allocate from; the location must be provided as an input to the API. However we are providing a basic auto allocation action which is mentioned in the API section.
- **Available quantity validations :** We are not currently placing any validations or reprocessing for scenarios where available quantities on product items change after the allocations are completed, due to modifications to quantities on hand.

Key decisions

Key Challenges	Decision	Signed off by
Data model	We initially planned to have a new entity for inventory allocation. As we came across Commerce Cloud's Inventory Reservation entities which looked similar and store quantity in the same granularity, we have decided to go with these entities with addition of few new fields.	Data Model Team, Commerce Cloud, SFS team

Milestone

Milestone	Release
260 - APIs, data model, Summary view UI, Processing layer of inventory states	**GA**
262 - extend allocation to support serialised and batch products. Extend API to support serialised products Experience Cloud support	**Pilot**
264+ - Extend API to support serialised products Extend API to support batch products Advanced UI, Support for serial products, Batch, Advanced auto allocation API and processing layer enhancements	

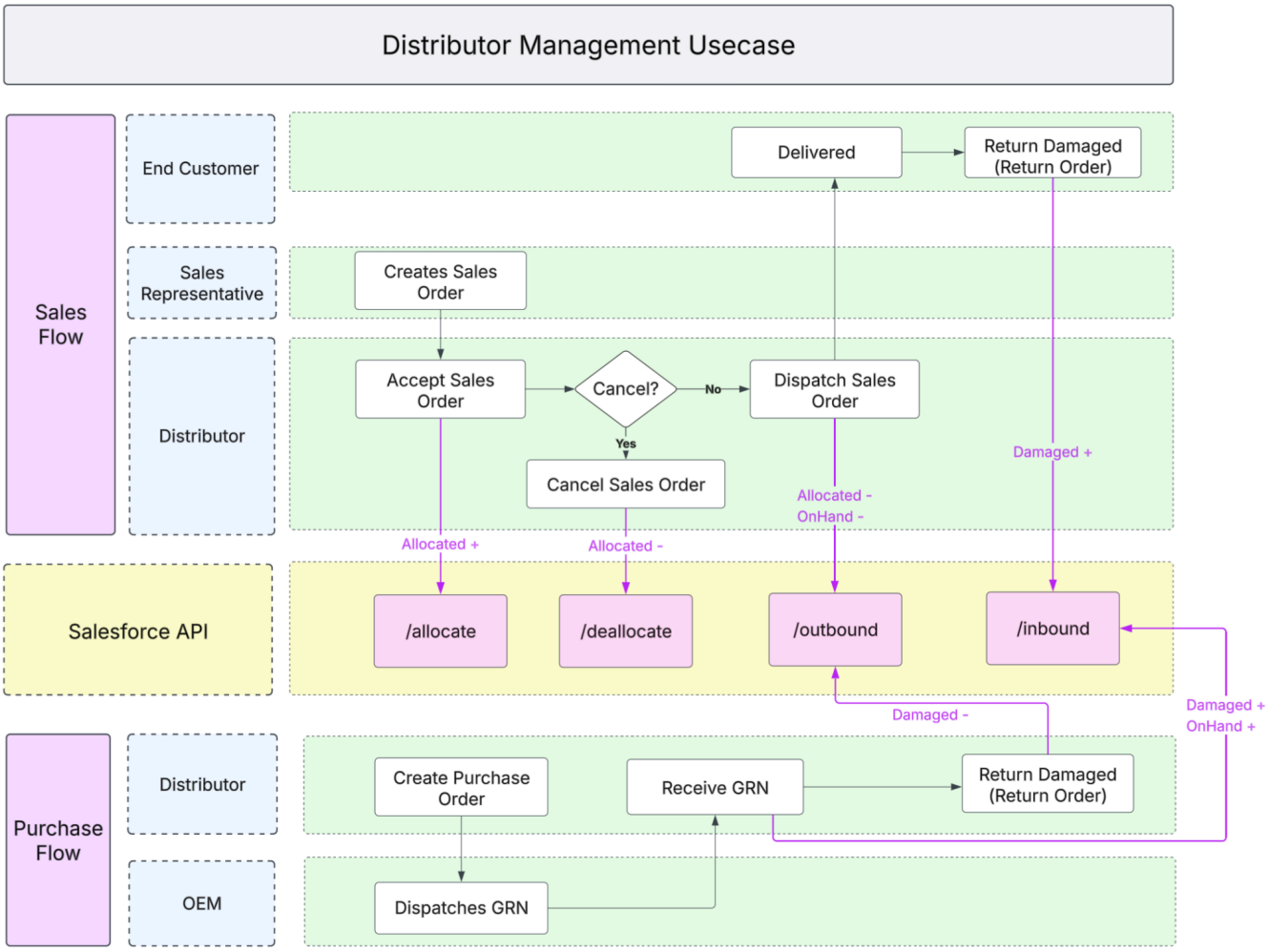
Technical Design

Persona

- **Sales Representative:** Manually allocates inventory when booking sales orders.
- **Field Technician:** Requires inventory allocation for work orders
- **Order Manager:** Oversees order fulfillment, may need to manually reallocate inventory between orders of different priorities.
- **System (e.g., DRO):** An automated order management system that programmatically initiates allocations.

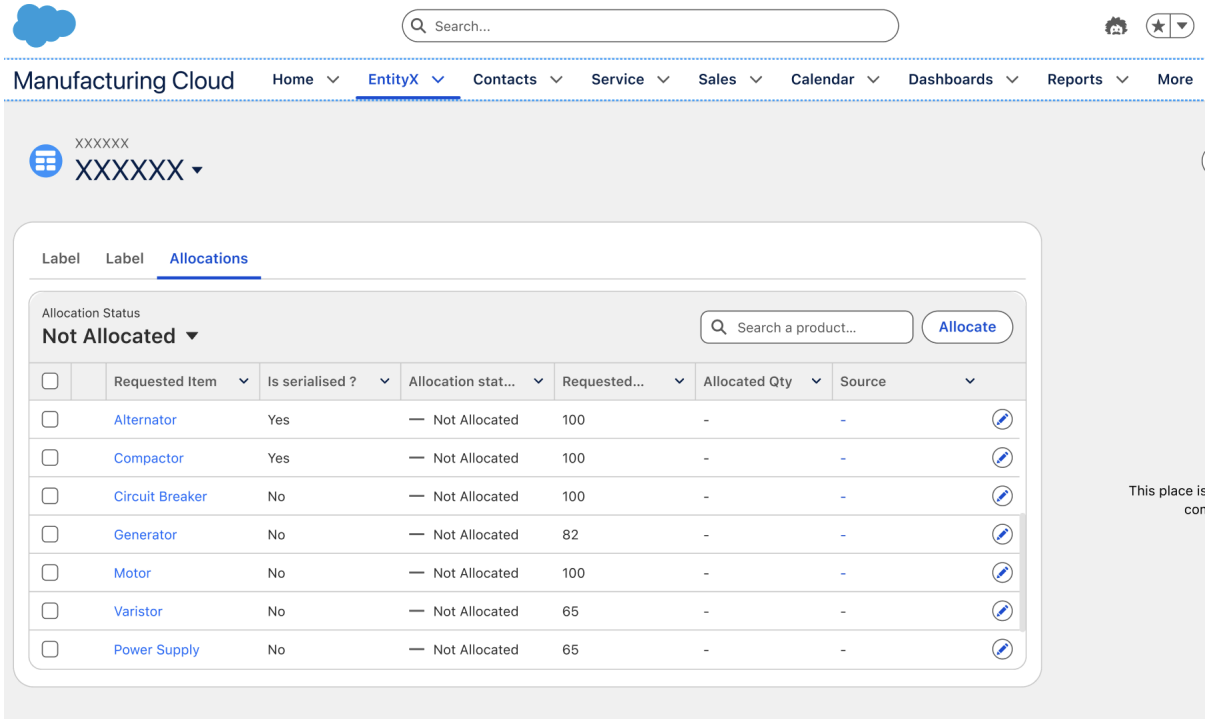
Flow Diagram (overall DMS scenario)

Note : *The below diagram gives an overall picture of inventory operations including allocation/deallocation. The scope of this design document is only for Allocation.*

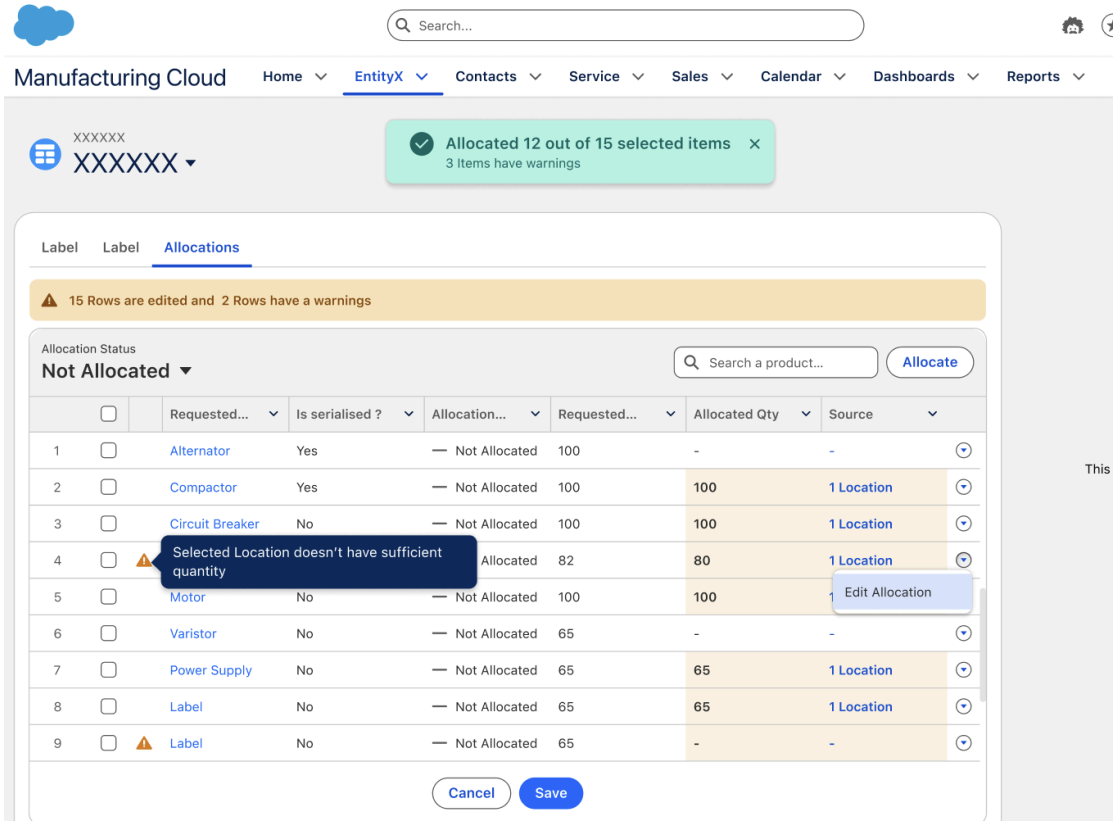


UI Layer (260 mocks) (262 mocks)

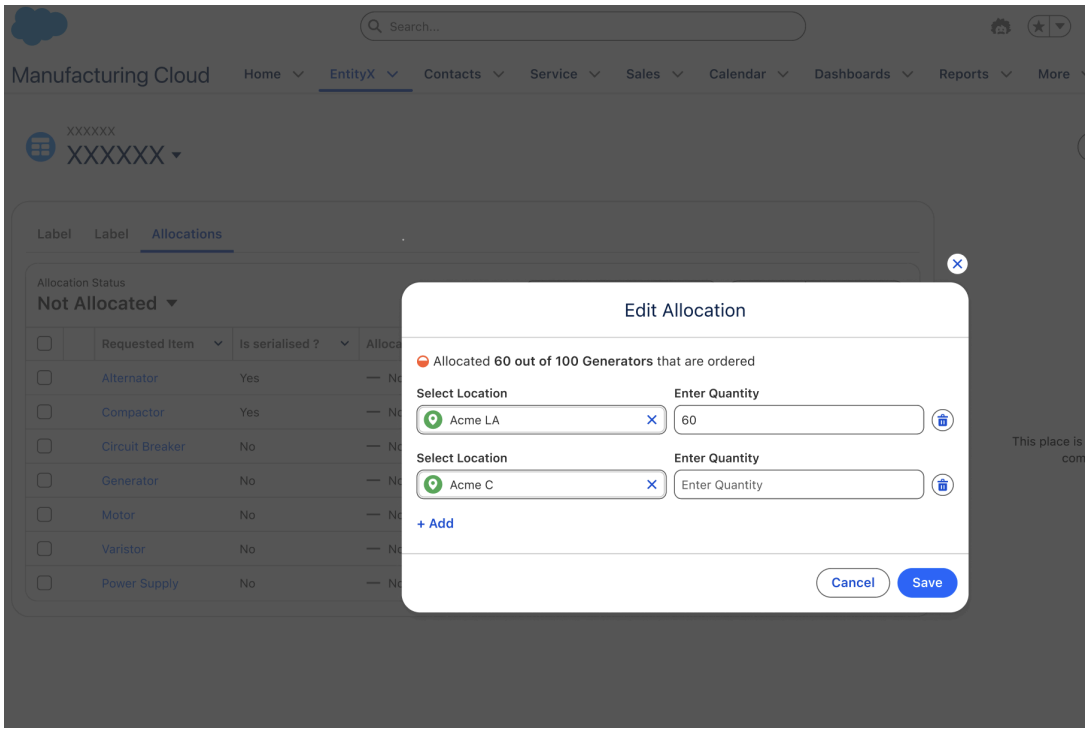
1. Order / Work Order/ Return Order page with the new Inventory Allocation LWC Component



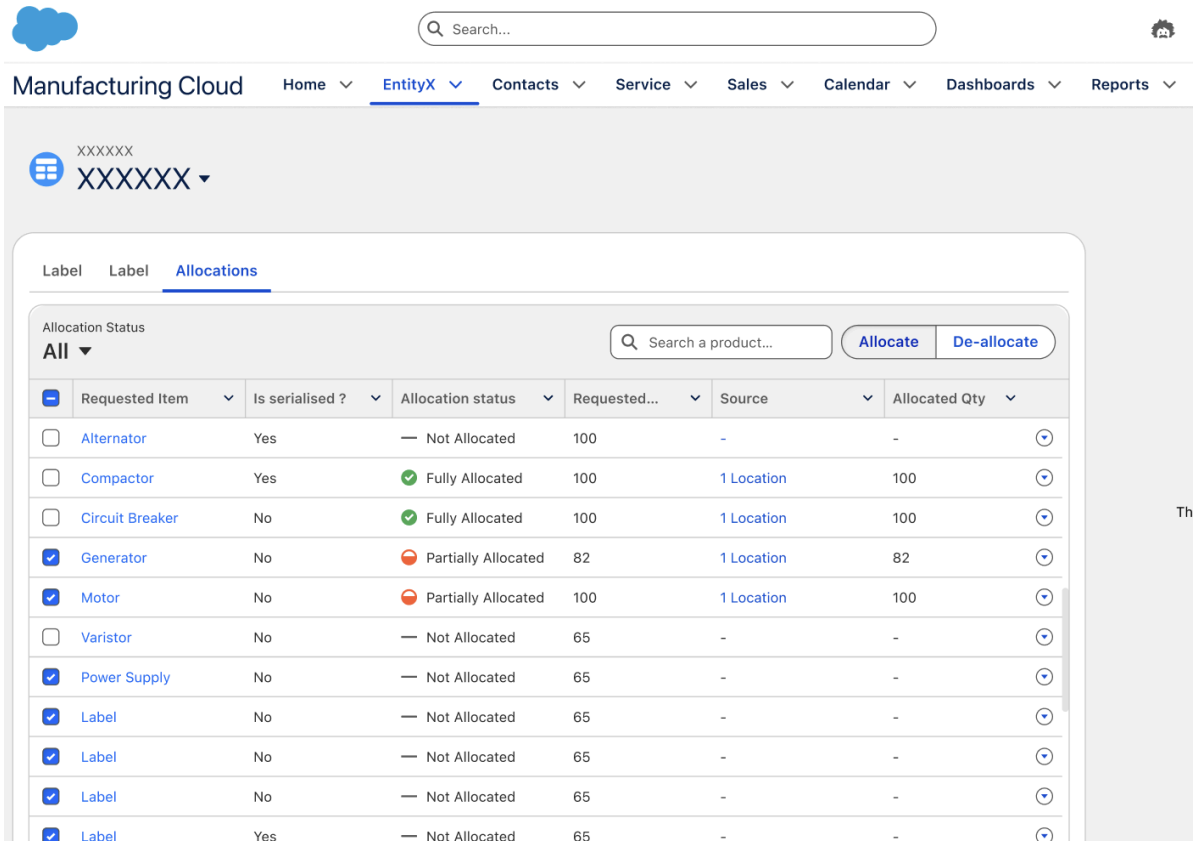
2. Allocate and Deallocate actions , proper messaging , pagination support and search by product



3. You can allocate quantities from multiple locations



4. Dynamic allocation status based on the allocated quantity vs required quantity



UI Highlights

- New Custom LWC component - Inventory Allocation
  - Made available on Order, Work Order, Return Order pages
  - Easily extensible to other Order entities. Just need few changes to our configuration files
- Ability to select multiple locations for a given product
- Ability to auto allocate , review and save.
- Ability to manually edit and save
- All the operations are asynchronous to handle bulk edits and in-app notifications are generated.
- Refresh button to re-load the latest inventory allocations since the allocation process is asynchronous

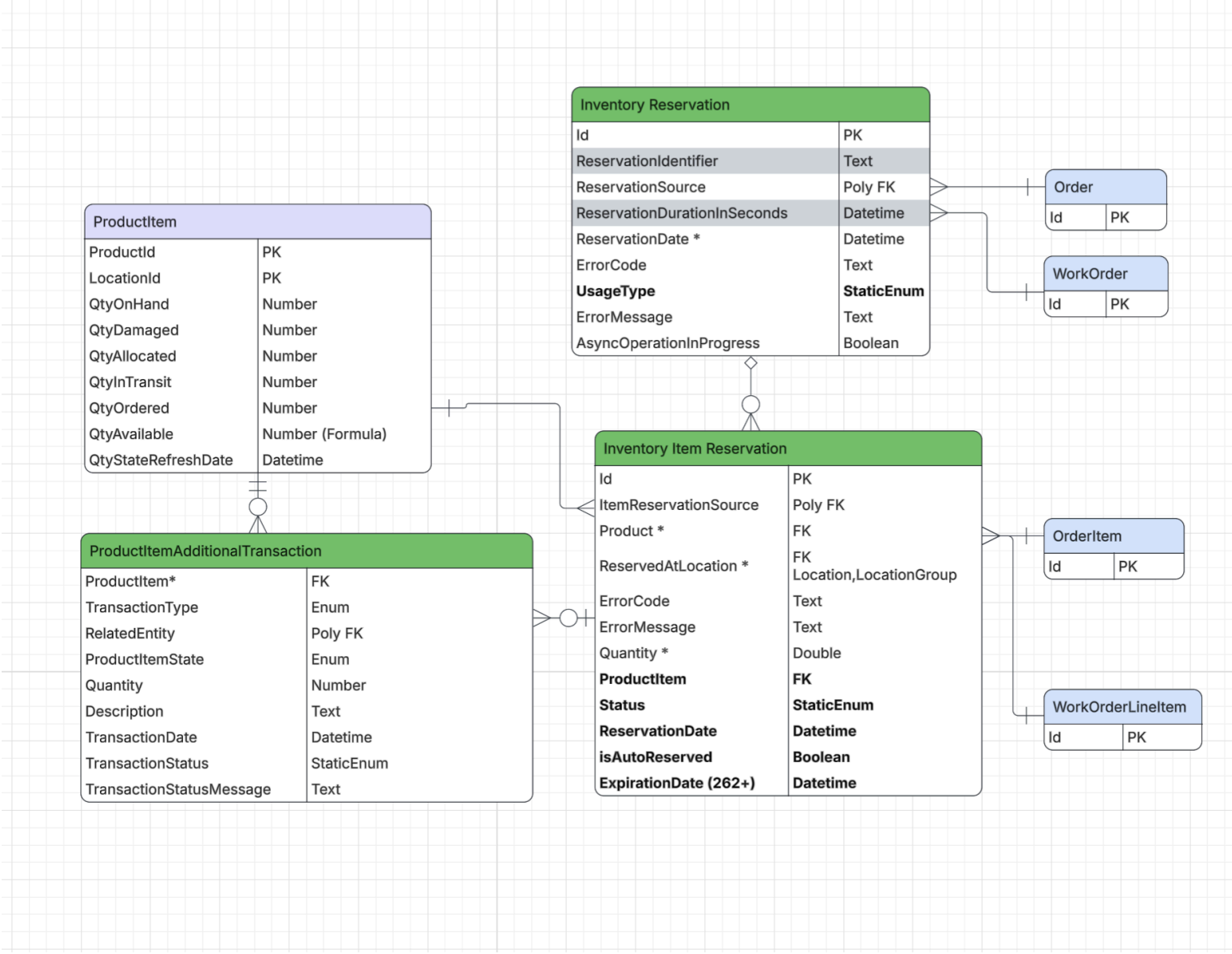
Logical/Conceptual Object Relationship Model (ERD)

Entity Details

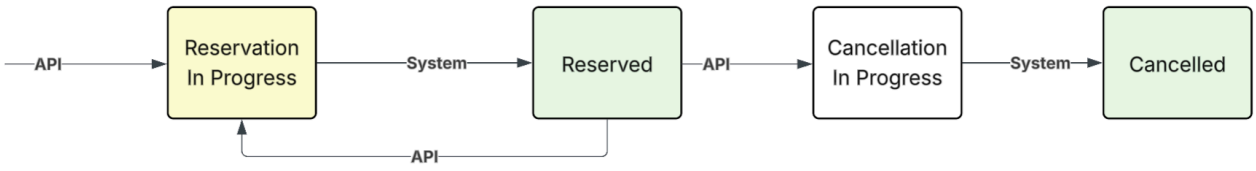
- We have reused the existing entities - Inventory Reservation and Inventory Item Reservation entities owned by Commerce Cloud.
  - Inventory Reservation - fields that are specific to Commerce Cloud -ReservationDurationInSeconds , Reservation Identifier will not be exposed to our users.
  - Added new fields -Status, ProductItem (FK), ReservationDate(at line item level), isAutoAllocated flag to Inventory Item Reservation entity.
  - Introduced Usage Type field on Inventory Reservation to identify the use case of the record. Usage type based validations will be implemented in the save hooks.

- For every create/update operation on Inventory Item Reservation, transaction entry is logged in Product Item Additional Transaction.
  - The initial status of the transaction will be ‘Pending’.
  - The Inventory processing layer will read the Pending transactions and update the allocated quantity on Product Item. The details of the end to end design are captured in the Architecture Section

ER Diagram



Status transitions of Item Reservation



*** Updates 262 (pilot) (refer 262 enhancements for details)

Batch Item Reservations

In the case of batched products, we should allow users to make batch based reservations to support use cases such as user prioritising reservation of batches that are going to expire first. To capture the batch item level reservations, we now have a new reservation entity ***"Inventory Batch Item Reservation"*** whose granularity is at "Product Batch Item" ie. Product + Location + Batch. This entity has a master-detail relation to InventoryItemReservation.

Other changes include introduction of **QuantityAllocated** and **QuantityAvailable** fields on "Product Batch Item" entity, similar to what we have done for "Product Item". We will capture the allocation quantity changes in Product Item Additional Transaction, for this we have added a new lookup field to ProductBatchItem in this entity.

Any increment/decrement to **QuantityAllocated** on Product Batch Item should increment/decrement the **QuantityAllocated** on Product item, similar to how it works for **QuantityOnHand.** The Allocation APIs will be extended to batch in future releases.

Serialized Product Reservations

In the case of serialised products, we should allow users to mention serialised products getting allocated. For this reason, we now have a new entity ***"Inventory Serialised Product Reservation"*** .This entity has a master-detail relation to InventoryItemReservation. We will capture the allocation quantity changes in the Product Item Additional Transaction entity.

We are introducing a new enum value " "***Allocated***" for Status field on Serialized Product.

Whenever serialized product status changes to Allocated, it should increase **QuantityAllocated** on Product Item and Product Batch Item (if applicable). The **QuantityAllocated** on Product Item (and Product Batch Item if applicable) should be equal to the number of Serialised Products in ***Allocated*** status.
The Allocation APIs are extended to serialised products

Alternate options explored for capturing allocation status on the Serialised product.

Challenge with current setup
The current Status enum values are [Available, Sent, Consumed, Damaged, Lost] . However, QuantityOnHand includes every status. But there are processes where delinking of product item is done alone with status change eg. Consumed.

Requirement
We need to support other status values such as [Allocated, InTransit, Ordered, Custom], and these statuses would add to new quantity states on product items. Eg. Allocated status should map to QuantityAllocated on ProductItem.

Comparison of various options to manage serialised product status

Option	Details	Pros	Cons / Risk
Add new value “Allocated” to existing enum - “SerialisedProductStatus”	Status moves to Allocated when a serialised product is allocated. It should move back to Available after deallocation. To support this, we should allow allocation only when the serialised product is in “Available” status (out of current available status)	Less confusion to the user since there will be only one status column to look into	We are not fully aware about the processes that set the current status values. Current customers reports might get impacted if they have done some reporting filtering on the status. When we add more status enum values such as ‘InTransit’, we won’t be able to have multiple status at the same time, because there is a requirement to allocate when the serialised product is in InTransit status.
Adding flag “isAllocated” to the Serialized Product	The flag will be toggled to true/false upon allocation/deallocation.	Safe option as it won’t impact existing functionality. We might still want to put validation that you could only allocate serialised products that are in “Available” status.	Extra boolean field. Less extensible as you won’t be able to add more enum values if a need arises.
Introducing a new static enum. Example - AllocationStatus **[Preferred option]**	AllocationStatus co-exists with existing Status enum. This enum holds only one value (Allocated) as per the current product requirement. We should introduce more enums eg. OrderStatus which can hold values that are related to a common process such as Ordered, InTransit	Inferring overall status is relatively easy as we group related values to different enums. Need some more thought on this approach. **(2)** Different static enums based on feature pref.	Need to introduce one new static enum for each category. Validations for various status permutations and combinations. Doesn’t impact current functionality.

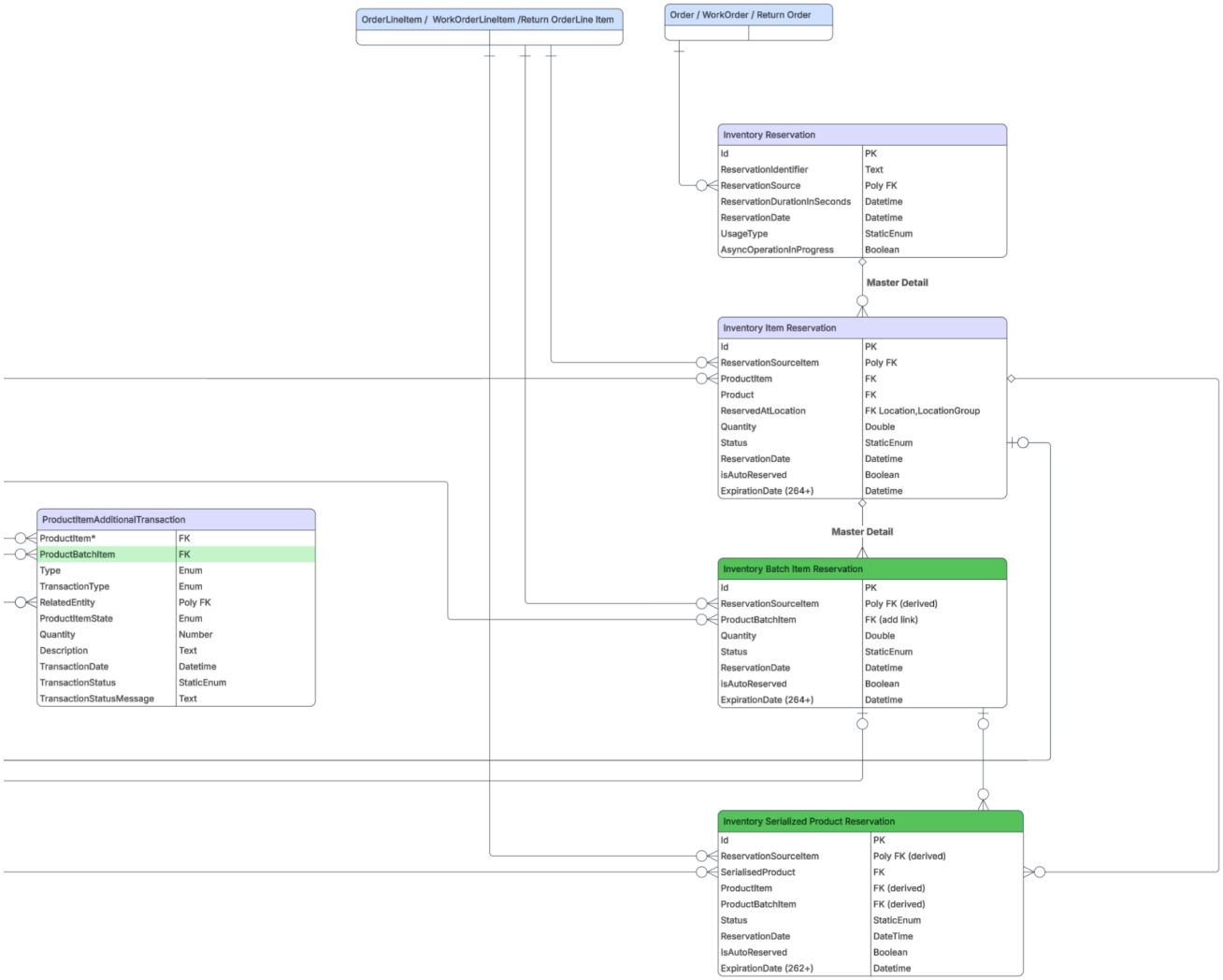
Conclusion:

Going with “AllocationStatus” new enum. Couple of benefits-

1. Going with this approach would not impact existing users.
2. We foresee cases where status values may overlap, so having a dedicated enum for the Reservation use case will keep things clean.

For detailed data model, please refer [Inventory Allocation 262 ERD]

Snapshot of the ERD



API Layer

Inventory Allocation API

POST connect/inventory/allocation

Request :

```
JSON
{

  "allocationData": [
    {
      "sourceId": "0AoSG000006cqWo0AI", //orderid
      "items": [
        {
          "sourceItemId": "0CoSG000006cqWo0AI", //orderlineitemid
          "allocations": [
            {
              "allocatedQuantity": "10",
              "sourceLocationId": "131SG000001ErblYAC"
            },
             {
              "allocatedQuantity": "20",
              "sourceLocationId": "131SG000001ErblYAD"
            }
          ],
          "deallocationLocationIds": [
            "131SG000001ErblYBA",
            "131SG000001ErblYCD"
          ]
        }
      ]
    }
  ]
}
```

Response : Includes standard response with error message if any

Inventory Deallocation API

POST connect/inventory/deallocation

Request :

```
None
{
  "deallocationData": [
    {
```

```
      "sourceId": "0AoSG000006cqWo0AI",
      "items": [
        {
          "sourceItemId": "0CoSG000006cqWo0AI",
          "sourceLocationIds": [
            "131SG000001ErblYAC",
            "131SG000001ErblYBC"
          ]
        }
      ]
    }
  ]
}
```

Response : Includes standard response with error message if any

Inventory Get Allocation API

Request :

GET connect/inventory/allocation/{orderId}?pageNumber=1&pageSize=10

Response :

JSON

```
{
  "allocationData": [
    {
      "lineItemId": "1WLxx0000000001GAA",
      "status":  "PARTIALLY_ALLOCATED",
      "totalRequiredQuantity" : 30,
      "totalRequestedAllocationQty": 20,
      "quantityAllocationPending": 10, -> this is quantity sum of pending transactions
      "quantityAllocationCompleted": 5, -> this is quantity sum of success transactions
      "requestedAllocations": [
        {
          "locationId": "131Ws000000kaV7IAI",
          "quantity": 8,
          "serialNumbers": [] //future scope
        }
      ],
      "allocations": [
        {
          "locationId": "131Ws000000kaV7IAI",
          "quantity": 12,
          "serialNumbers": [] //future scope
        }
      ]
```

```
    }
  ],
  "meta": {
    "pageNumber": 1,
    "pageSize": 10,
    "hasNext" : true
  }
}
```


API details

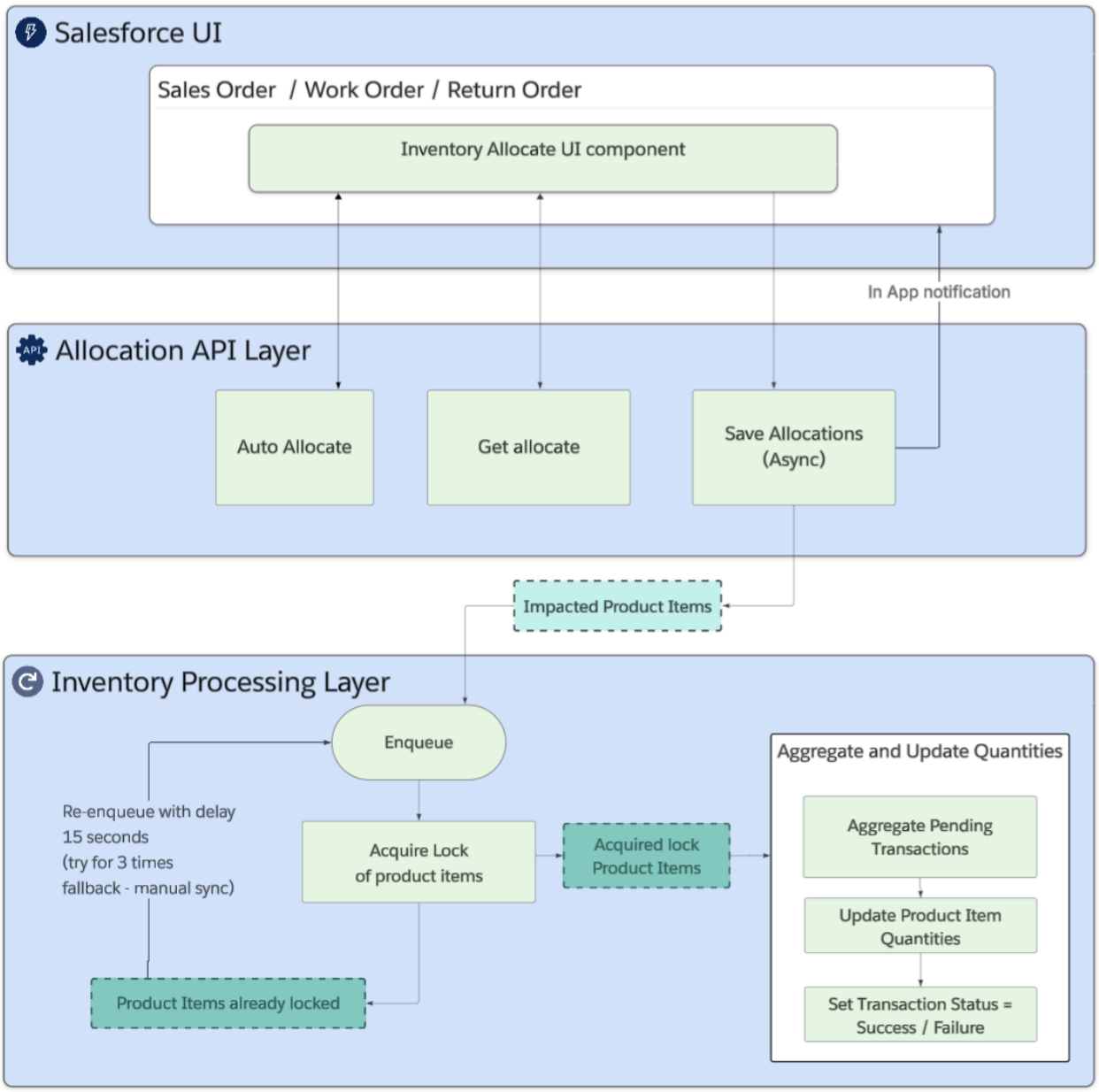
- This API provides details about the allocation status which takes these values
  - Not allocated
  - Partially allocated
  - Fully allocated
- It gives allocation summary details for each line item
  - Total required quantity -> this is the line item quantity
  - Total requested allocation quantity -> this is the total allocation quantity requested for the line item
  - Total quantity allocation completed - this is the total quantity successfully allocated from the locations selected
- To get more information about the individual transaction status, user can go to related list of Product item Additional Transactions under Inventory Item Reservation

Inventory Auto Allocation

- Pre-requisite
  - User need to map Order's location with the desired inventory location for all the products by creating records in the entity - 'Product Fulfilment Location'

- Allocation action
  - When a user performs the allocation action, we use the data from 'Product Fulfilment Location' and the user is shown the preview of allocation.
  - Users can review, edit the allocation data that's auto filled before performing the save.
  - Currently allocation action is UI only action, but we plan to build a dedicated API in future releases with advanced capabilities such as supporting multiple source locations

Architecture



Key highlights

- ❖ Inventory Processing Layer is designed not just for Allocation use case
  - ➢ It is designed to support all kinds of quantity states , not just for allocation.
  - ➢ When we build features for incoming inventory involving other quantity states such as In-Transit, Ordered ,the inventory processing layer can be used to update the corresponding quantities on the Product Item.

Alternate Designs Considered for processing layer

Approach	Description	Pros	Cons
Message Queue **(Finalised approach)**	Push MQ message with list of impacted product items	1.Allows re-queuing of product items for which we couldn't acquire lock in the handler. This is achieved using message.setEnqueueDelay(15); We will have a cap on number of re-enqueues 2. This guarantees the messages are processed instantly when there is less traffic or no concurrency and we will have option to retry after some time without too much delay 3. Process multiple product items at once. We decided to publish one MQ message per bunch of impacted product items (kept the number as 10 for now). We will finalize the number of product items to process in one message handler after performance testing of the handler.	1. MQ processing time is not guaranteed
Fixed Thread Pool (seam stress) to manage thread pool	Initiate a fixed thread pool and for each impacted product item submit a thread	1. We can do the performance testing and come up with optimal thread pool size. 2. Processing time is predictable	1. We would be responsible for managing the thread pool. 2. There is a high chance that this will impact other processes running on the app server 3. We won't be able to retry with delay.
Cron job	Run job that processes transactions for all impacted product items in some fixed time interval	1. Easy to manage the job and can be run on demand	1. Significant delay since the allocated quantity is not updated till the job is run.
Platform event	Publish platform event for each product item	Similar to MQ, where we will have events per impacted product item.	1. Doesn't have capability to retry after some delay when we are unable

			to acquire lock 2. We need to invest in a new platform event type.

Non-functional requirement (NFR)

High concurrency

We have to support high scale scenarios as there can be allocations happening for multiple products and locations at the same time. The API and allocation actions must be performant to handle a large number of transactions without significant delays.

To handle high volumes, we have decided to avoid direct updates on Product Item for various quantity states. The Inventory Processing layer minimises the updates to Product Item to large extent by aggregating the pending transactions and doing a single update to Product Item.

Risks

When there are high volumes of allocations happening at the same time, updates to product item quantities can get delayed as they are handled asynchronously in the MQ handler.

We have added 'Quantity State Last Refresh Date' to the Product Item entity to give visibility to users when the quantities are updated last and to get more details about the transactions, users can look into transaction status and transaction status messages that are captured in Product Item Additional Transaction.

Licensing

1. New platform (***Inventory Allocation***) and user licences (***Inventory Allocation User***) that will be bundled under Inventory Search And Transfer Addon (existing addon)
    a. Org perm : InventoryAllocation
    b. Org preference : InventoryAllocationEnabled
    c. Platform license : Inventory Allocation (InventoryAllocation)
    d. User perm: ManageInventoryAllocation
    e. User license : Inventory Allocation User (InvAllocationUserPsl)
2. Separate Setup page ***(Inventory Allocation Settings)*** to to toggle the preference
3. All the APIs , UI will be behind the new access checks
    a. orgHasInventoryAllocationEnabled
    b. userHasAccessToInventoryAllocation

Security

WIP

Patentability

[What aspects of this might be patentable?]

References

260 Epic Document: Inventory Status & Inventory Allocation
258 DMS Inventory Management - Design Document

POCs & Spikes

ProductitemAdditionalTransaction processing using threads

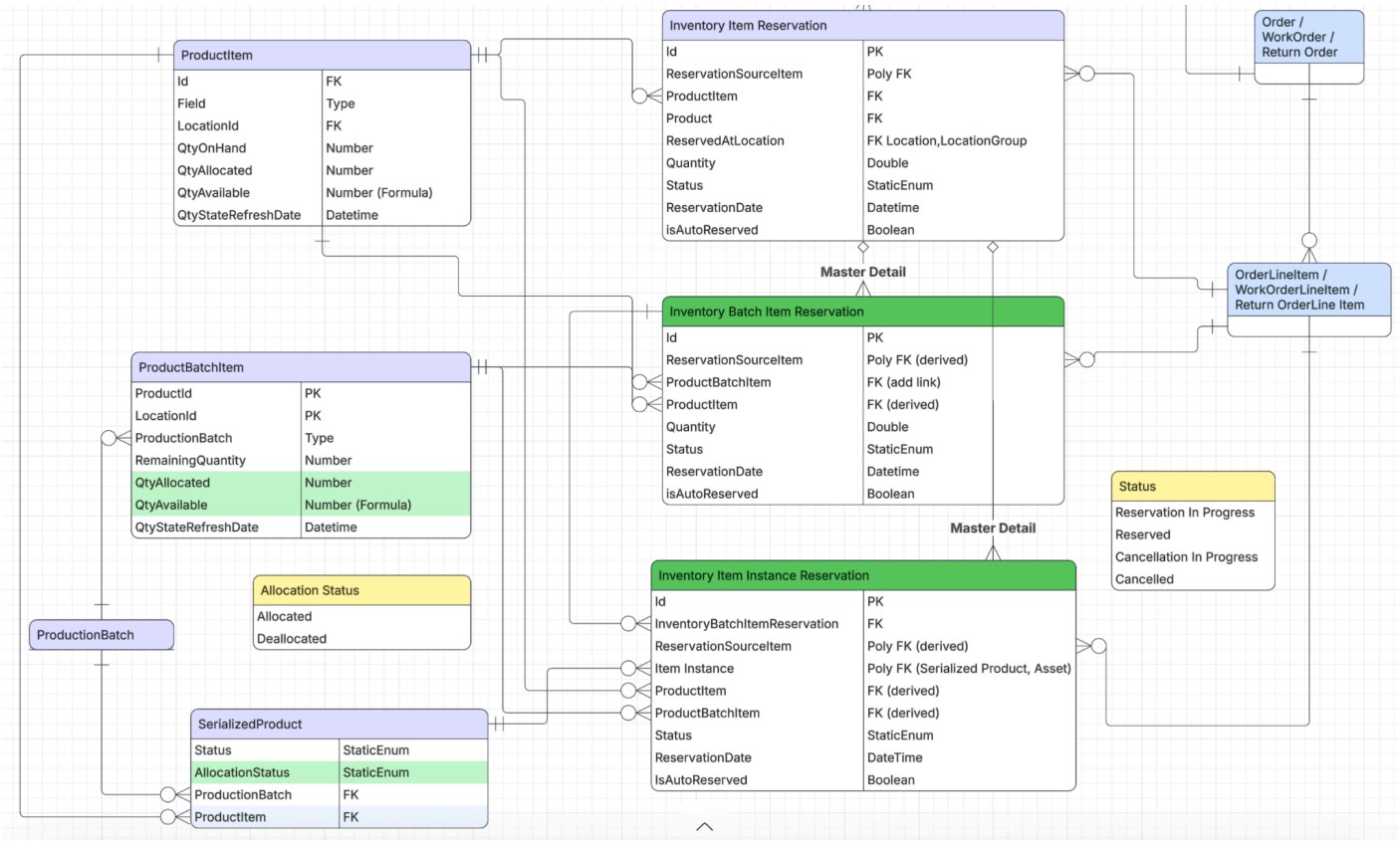
Q3Q4 Discussion Points

1. **How are custom enums for the** `transactionType` field within `ProductItemAdditionalTransactions` being handled?
    a. Users will choose transaction type as other and will mention the transaction type in a new dynamic enum to capture custom status. However it's the users responsibility to read the transactions and update the custom status quantity on product items.
2. What is the process for populating the `ProductItem` and `ProductItemAdditionalTransaction` entities from day zero?
    a. Users can opt to feed product item transactions and write an aggregation job that writes to Product Item. Subsequent updates will be handled by our API

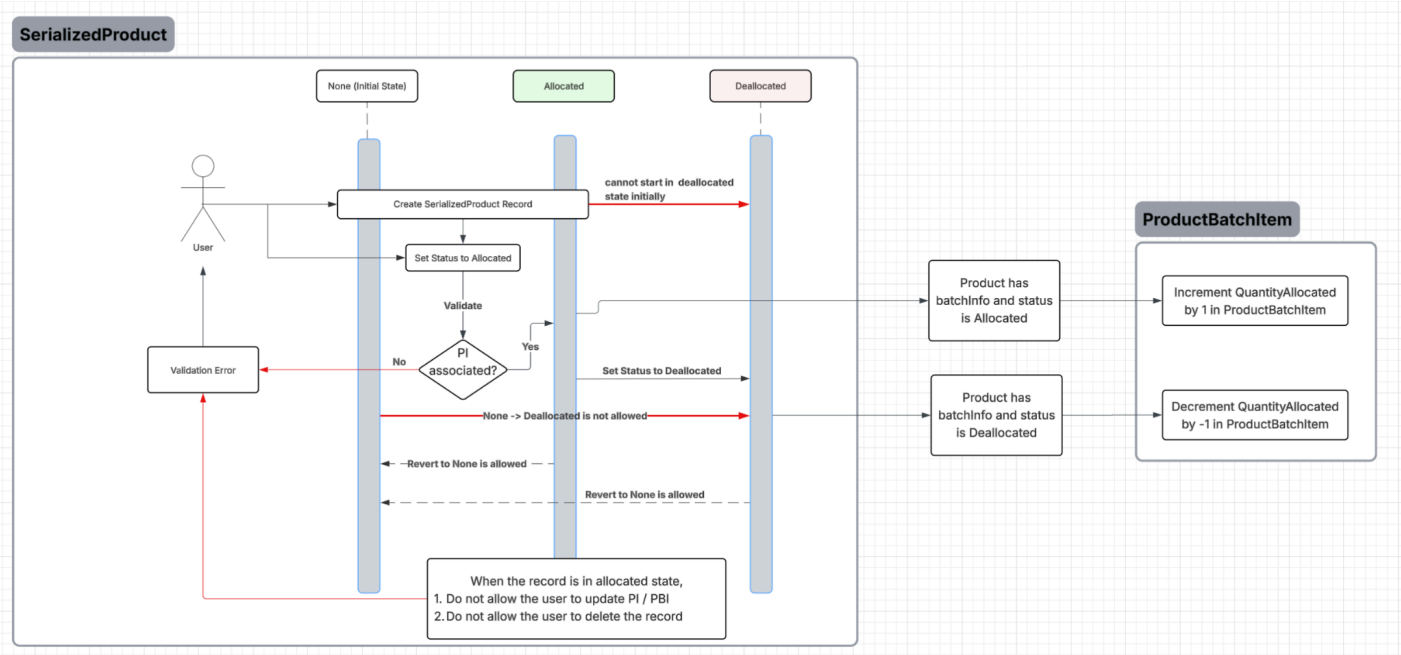
262 Enhancements

InventoryAllocation Enhancements (262)

Datamodel



AllocatedStatus Transitions & Validations



New and Updated Data Model Changes

1. SerializedProduct

- AllocationStatus (Enum) - This field represents whether a serialized product instance is currently allocated to a reservation.
  Supported values: None (Initial state), Allocated, Deallocated

2. InventoryBatchItemReservation (New Entity)

- Represents batch-level inventory reservations.
- Created as a child (master-detail) of InventoryItemReservation.
- Key responsibilities:
  - Tracks reserved quantity at the batch level.
  - Links reservation source items (Order / Work Order / Return Order line items).
  - Drives allocation and deallocation updates on ProductBatchItem.
  - Quantity Synchronization:
    Whenever an InventoryBatchItemReservation record is created, updated, or deleted, the system recalculates the reserved quantity and updates the parent quantity in InventoryItemReservation to keep batch and item-level reservations in sync.

3. InventoryItemInstanceReservation (New Entity)

- Represents instance-level (serialized product) reservations.
- Created as a child (master-detail) of InventoryItemReservation.

- Key responsibilities:
    - Associates a specific SerializedProduct with a reservation.
    - Ensures one-to-one tracking between instance reservation and serialized inventory.
    - Quantity Synchronization:
      Whenever an InventoryItemInstanceReservation record is created, updated, or deleted, the system recalculates the reserved quantity and updates the parent quantity in InventoryItemReservation to ensure consistency across instance, batch, and item-level reservations.

4. ProductBatchItem
- New fields added:
    - QtyAllocated
    - QtyAvailable (Formula)
- These fields track allocation counts at the batch level.
- Quantity updates are driven by allocation and deallocation events from reservations.

5. ProductItemAdditionalTransaction
- New fields added : ProductBatchItem

API Layer

Allocation api

Inventory Allocation 262 [Final] - Connect API

The API supports the following allocation patterns. Existing behavior from v260 remains unchanged where noted.

1. Location-level Allocation (Non-batched, Non-serialized) - Exists in v260

Exists in v260
- Allocates inventory directly at the location level.
- No batch or serialized product information is involved.
- Creates an InventoryItemReservation record for the given productId and locationId (productItemId).

```JSON
{
"sourceLocationId": "000xxx00", //Chennai
"allocatedQuantity": 10
}
```

2. Batch-level Allocation (Non-serialized)

Description:

- Allocates inventory by batches without serialized products.

Required Fields:

- batchItemId
- quantity

Behavior:

- Creates an InventoryItemReservation record for the given productId and locationId.
- Creates one InventoryBatchItemReservation record per batchItemId.
- Establishes a 1:N relationship:
    - InventoryItemReservation → n number of InventoryBatchItemReservation

```JSON
{
"batchItemId": "000xxx00", //PBI-1
"quantity": 10
}
```

3. Serialized Product – Batch Allocation

Description:

- Allocates serialized products that belong to a specific batch or batch item.

Rules

- Either batchId or batchItemId is mandatory.
- batchedSerializedProductIds is mandatory.
- quantity is optional (derived from serialized product count if omitted).

Behavior:

- Creates an InventoryItemReservation record for the given productId and locationId.
- Creates an InventoryBatchItemReservation for each batchId or batchItemId.
- Creates an InventorySerializedProductReservation for each serialized product.
- Relationships:
    - InventoryItemReservation → InventoryBatchItemReservation (1:N)

- InventoryBatchItemReservation → InventorySerializedProductReservation (1:N)

Note: When batchId is provided, batchItemId is derived using batchId + location.

```
JSON
{
"batchItemId": "000x00", // PBI-1
"quantity": 2,
"batchedSerializedProductIds": ["000x00", "000x00"] // SP-4, SP-6
}
{
"batchId": "000x00", // PB-1
"quantity": 2,
"batchedSerializedProductIds": ["000x00", "000x00"] // SP-4, SP-6
}
```

4. Serialized Product – Non-batch Allocation

Description:

- Allocates serialized products that are not associated with any batch.

Required Fields:

- unbatchedSerializedProductIds

Behavior:

- Creates an InventoryItemReservation record for the given productId and locationId.
- Creates an InventorySerializedProductReservation record for each serialized product.
- Relationship:
  - InventoryItemReservation → InventorySerializedProductReservation (1:N)

```
JSON
{
"unbatchedSerializedProductIds": ["000x00", "000x00"] // SP-4, SP-6
}
```

Payload

```
JSON
{
  "allocationData": [
    {
```

```
"sourceId": "0AoSG000006cqWo0AI",
"items": [
  {
    //1.by locations (non-batched and non-serialized)
    "sourceItemId": "00xxx00", // Generator
    "allocations": [
      {
        "sourceLocationId": "00xxx00", // Chennai
        "allocatedQuantity": 75
      },
      {
        "sourceLocationId": "00xxx00", // Surat
        "allocatedQuantity": 23
      }
    ]
  },
  {
    //2.by batches (non-serialized)
    "sourceItemId": "00xxx00", // Motor
    "allocations": [
      {
        "sourceLocationId": "00xxx00", // Bangalore
        "allocatedQuantity": 30, // optional
        "batches": [
          {
            "batchItemId": "00xxx00", // PBI-1
            "quantity": 10
          },
          {
            "batchItemId": "00xxx00", // PBI-2
            "quantity": 20
          }
        ]
      }
    ]
  },
```

```
        {
          // 3. by batches (serialized)
          "sourceItemId": "00xxx00", // Circuit
          "allocations": [
            {
              "sourceLocationId": "00xxx00", // Bangalore
              "allocatedQuantity": 4,  // optional
              "batches": [
                {
                  "batchId": "00xxx00", // PB-1
                  "quantity": 1, // optional
                  "batchedSerializedProductIds": ["00xxx00"]
                },
                {
                  "batchItemId": "00xxx00", // PBI-12
                  "quantity": 3,  // optional
                  "batchedSerializedProductIds":
["00xxx00","00xxx00","00xxx00"]
                }
              ]
            }
          ]
        },
        {
          // 4. by seralized non-batched
          "sourceItemId": "00xxx00", // Iphone
          "allocations": [
            {
              "sourceLocationId": "00xxx00", // Hyderabad
              "allocatedQuantity": 3,  // optional
              "unbatchedSerializedProductIds":
["00xxx00","00xxx00","00xxx00"]
            }
          ]
        }
      ]
```

```
    }
  ]

}
```

262 scope

JSON

```
{
  "allocationData": [
    {
      "sourceId": "0AoSG000006cqWo0AI",
      "items": [
        {
          //by locations (non-batched and non-serialized)
          "sourceItemId": "00xxx00", // Generator
          "allocations": [
            {
              "sourceLocationId": "00xxx00", // Chennai
              "allocatedQuantity": 75
            },
            {
              "sourceLocationId": "00xxx00", // Surat
              "allocatedQuantity": 23
            }
          ]
        },
        {
          // by serialized non-batched
          "sourceItemId": "00xxx00", // Iphone
          "allocations": [
            {
              "sourceLocationId": "00xxx00", // Hyderabad
              "allocatedQuantity": 3,  // optional
```

```
              "unbatchedSerializedProductIds":
["00xxx00","00xxx00","00xxx00"]
            }
          ]
        }
      ]
    }
  ]

}
```

Deallocation api

Inventory Deallocation 262 [Final] - Connect API

The API supports the following deallocation patterns using a nested request structure.

- If only sourceItemId is provided, the item is fully deallocated.
- If sourceLocationId is provided, deallocation is scoped to that location.
- Providing batchItemIds or serializedProductIds further narrows the deallocation scope.
- Mixed deallocation types are supported within the same request.

Internal logic

- Deallocation is performed by updating matching reservation records and setting their Status to Cancelled.
- Child reservations (batch-level and serialized-level) are cancelled based on the specificity of the request.

1. Item-level Deallocation - Exists in v260

Description:

- Deallocates the item completely across all locations, batches, and serialized products.

Behavior:

- All reservations matching the given sourceItemId are cancelled.

2. Location-level Deallocation - Exists in v260

Description:

- Deallocates all reservations for an item at one or more locations.

Behavior:

- Cancels all reservations for the given item at the specified location(s).

3. Batch-level Deallocation (Non-serialized)

Description:

- Deallocates reservations for specific batch items without serialized products.

Required Fields:

- batchItemIds

Behavior:

- Cancels batch-level reservations for the specified batch items at a given location.

4. Batch-level Deallocation (Serialized)

Description:

- Deallocates reservations for serialized products that belong to a batch item.

Required Fields:

- batchItemIds
- serializedProductIds

Behavior:

- Cancels serialized product reservations scoped to the specified batch items and location.

5. Serialized Product Deallocation (Non-batched)

Description:

- Deallocates reservations for serialized products that are not associated with any batch.

Required Fields:

- serializedProductIds

Behavior:

- Cancels serialized product reservations at the specified location.

Payload

```JSON
{
  "deallocationData": [
    {
      "sourceId": "00xxx00", // Reservation Source
      "items": [
        {
          //1.by item – deallocate the item completely
          "sourceItemId": "00xxx00" // Generator
        },
        {
          //2.by location – deallocate all records at specific locations
          "sourceItemId": "00xxx00",
          "sourceLocationIds": ["00xxx00","00xxx00"] // support 260 backward
      compatibility
```

```
},
{
  //2.by location – deallocate all records at specific locations
  "sourceItemId": "00xxx00", // Motor
  "deallocations": [
    {
      "sourceLocationId": "00xxx00" // Surat
    },
    {
      "sourceLocationId": "00xxx00" // Chennai
    }
  ]
},
{
  //3.by batches (non-serialized)
  "sourceItemId": "00xxx00", // Headlight
  "deallocations": [
    {
      "sourceLocationId": "00xxx00", // Surat
      "batchItemIds": [
        "00xxx00", // PBI-1
        "00xxx00", // PBI-33
        "00xxx00"  // PBI-22
      ]
    }
  ]
},
{
  //4.by serialized products (batched or non-batched)
  "sourceItemId": "00xxx00", // Headlamp
  "deallocations": [
    {
      "sourceLocationId": "00xxx00", // Bangalore
      "serializedProductIds": [
        "00xxx00", // spid22
        "00xxx00"  // spid23
      ]
    }
  ]
}
```

```
      ]
    }
  ]
}
```

262 Scope

JSON

```
{
  "deallocationData": [
    {
      "sourceId": "00xxx00", // Reservation Source
      "items": [
        {
          //by item – deallocate the item completely
          "sourceItemId": "00xxx00" // Generator
        },
        {
          //by location – deallocate all records at specific locations
          "sourceItemId": "00xxx00",
          "sourceLocationIds": ["00xxx00","00xxx00"] // support 260 backward
      compatibility
        },
        {
          //by location – deallocate all records at specific locations
          "sourceItemId": "00xxx00", // Motor
          "deallocations": [
            {
              "sourceLocationId": "00xxx00" // Surat
            },
            {
              "sourceLocationId": "00xxx00" // Chennai
            }
          ]
        },
```

```
        {
          //by serialized products (batched or non-batched)
          "sourceItemId": "00xxx00", // Headlamp
          "deallocations": [
            {
              "sourceLocationId": "00xxx00", // Bangalore
              "serializedProductIds": [
                "00xxx00", // spid22
                "00xxx00"  // spid23
              ]
            }
          ]
        }
      ]
    }
  ]
}
```

Get allocation api

Get Allocation api 2.0

Scenario 1: Regular Product (No Batches, No Serialization) - Exists in v260

```
JSON
{
  "allocations": [
    {
      "locationId": "loc123",
      "locationName": "Warehouse A",
      "reservedQuantity": 50.0,
      "availableQuantity": 200.0
    }
  ]
}
```

Scenario 2: Batched Product (No Serialization)

JSON

```
{
  "allocations": [
    {
      "locationId": "loc123",
      "locationName": "Warehouse A",
      "reservedQuantity": 50.0,
      "availableQuantity": 200.0,
      "batchReservations": [
        {
          "batchReservationId": "batchRes123",
          "productionBatch": "BATCH-001",
          "reservedQuantity": 30.0,
          "status": "Allocated",
          "instanceReservations": []  // No serialization
        },
        {
          "batchReservationId": "batchRes456",
          "productionBatch": "BATCH-002",
          "reservedQuantity": 20.0,
          "status": "Allocated",
          "instanceReservations": []
        }
      ],
      "instanceReservations": null
    }
  ]
}
```

Scenario 3: Serialized Product (No Batches)

JSON

```
{
  "allocations": [
    {
      "locationId": "loc123",
      "locationName": "Warehouse A",
      "reservedQuantity": 3.0,
      "availableQuantity": 10.0,
      "batchReservations": null,
      "instanceReservations": [
        {
          "instanceReservationId": "instRes123",
```

```
          "itemInstanceId": "serial123",
          "itemInstanceType": "SerializedProduct",
          "serialNumber": "SN-12345",
          "status": "Allocated"
        },
        {
          "instanceReservationId": "instRes456",
          "itemInstanceId": "serial456",
          "itemInstanceType": "SerializedProduct",
          "serialNumber": "SN-67890",
          "status": "Allocated"
        }
      ]
    }
  ]
}
```

Scenario 4: Batched + Serialized Product

JSON

```
{
  "allocations": [
    {
      "locationId": "loc123",
      "locationName": "Warehouse A",
      "reservedQuantity": 5.0,
      "availableQuantity": 20.0,
      "batchReservations": [
        {
          "batchReservationId": "batchRes123",
          "productionBatch": "BATCH-001",
          "reservedQuantity": 3.0,
          "status": "Allocated",
          "instanceReservations": [
            {
              "instanceReservationId": "instRes123",
              "itemInstanceId": "serial123",
              "itemInstanceType": "SerializedProduct",
              "serialNumber": "SN-12345",
              "status": "Allocated"
            },
            {
```

```
            "instanceReservationId": "instRes456",
            "itemInstanceId": "serial456",
            "itemInstanceType": "SerializedProduct",
            "serialNumber": "SN-67890",
            "status": "Allocated"
          }
        ]
      }
    ],
    "instanceReservations": [
      {
        "instanceReservationId": "instRes789",
        "itemInstanceId": "serial789",
        "itemInstanceType": "SerializedProduct",
        "serialNumber": "SN-11111",
        "status": "Allocated"
      }
    ]
  }
 ]
}
```

API Enhancement in 262

JSON

```
{
  "allocations": [
    {
      "locationId": "loc123",
      "locationName": "Warehouse A",
      "reservedQuantity": 3.0,
      "availableQuantity": 10.0,
      "instanceReservations": [
        {
          "instanceReservationId": "instRes123",
          "itemInstanceId": "serial123",
          "itemInstanceType": "SerializedProduct",
          "serialNumber": "SN-12345",
          "status": "Allocated"
        },
        {
```

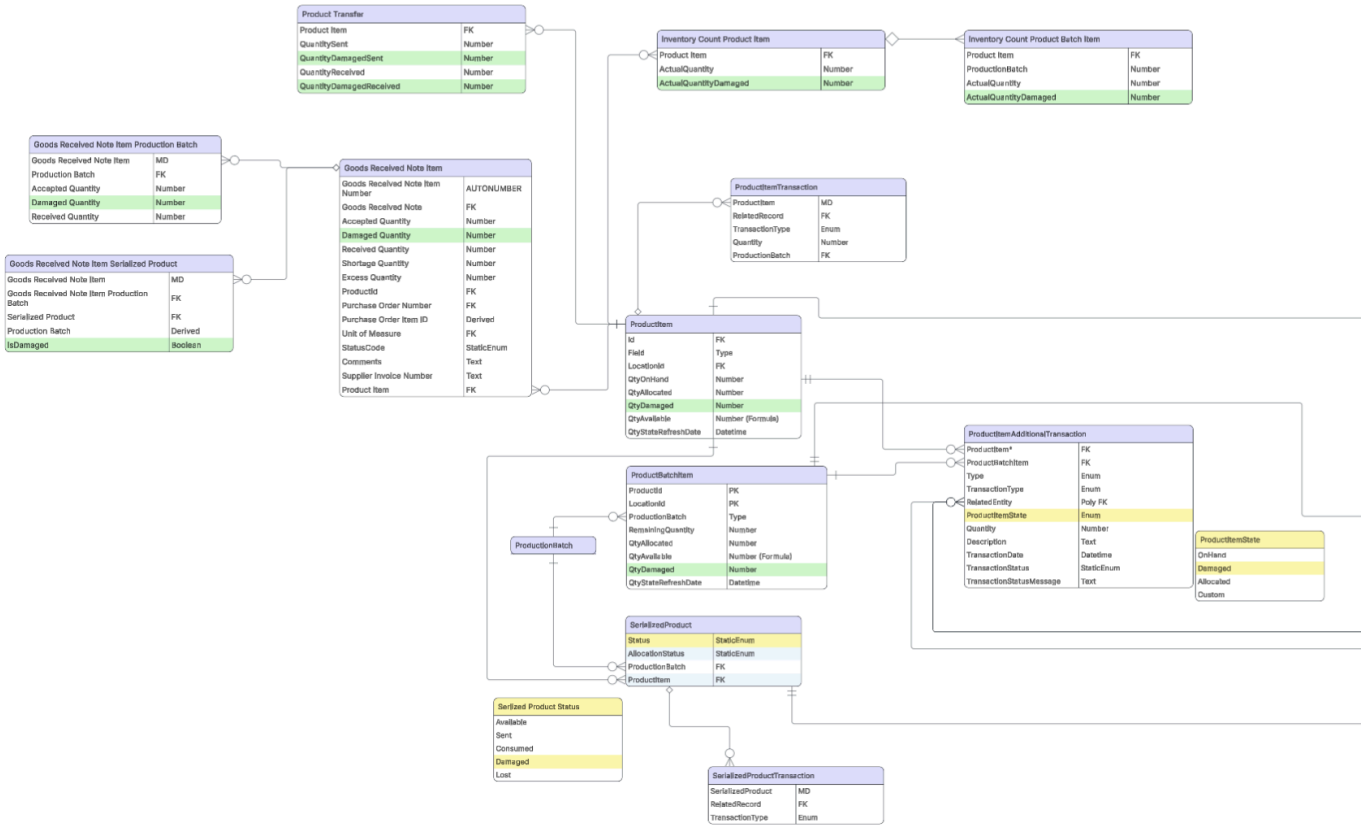
```
          "instanceReservationId": "instRes456",
          "itemInstanceId": "serial456",
          "itemInstanceType": "SerializedProduct",
          "serialNumber": "SN-67890",
          "status": "Allocated"
        }
      ]
    }
  ]
}
```

264 - Damaged Quantity

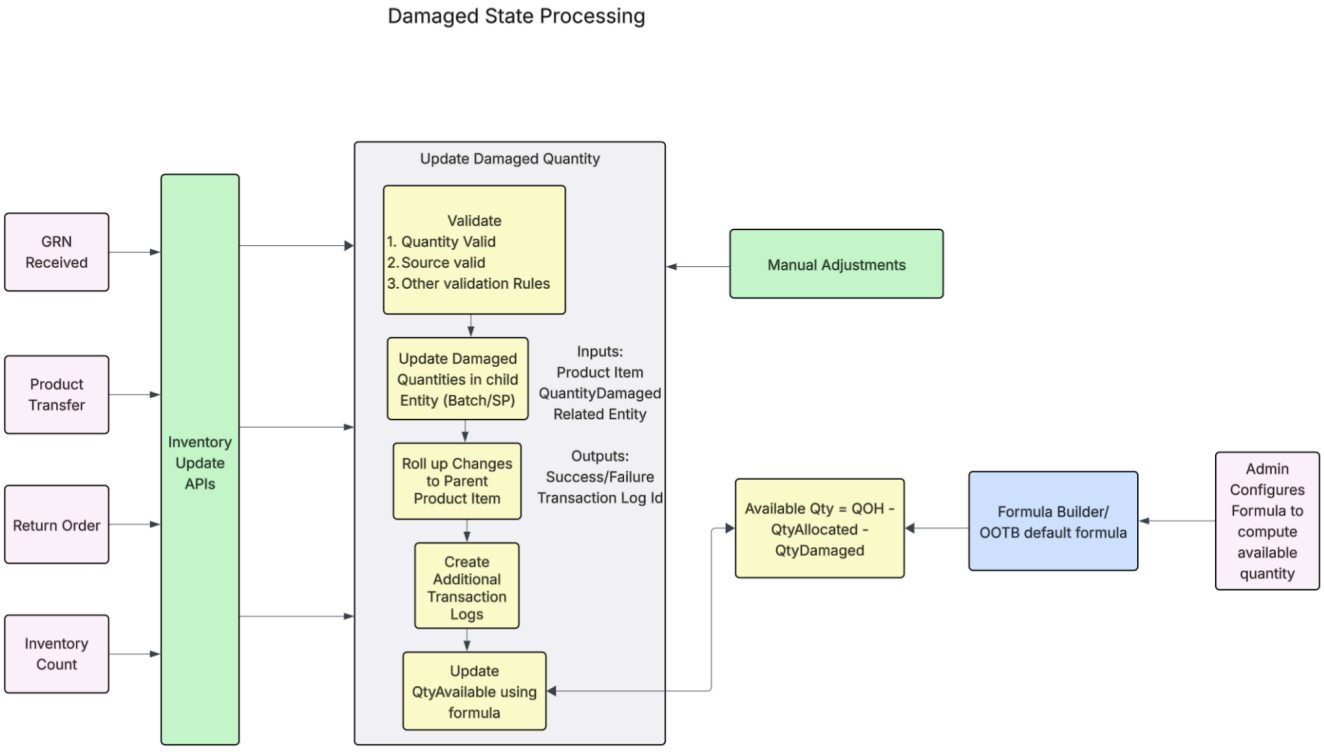
Damaged Quantity Processing

Requirements: [Damaged Quantity Processing PRD]

Data Model Enhancements:



[Lucid Link]


Damaged State Processing
Update Damaged Quantity
GRN Received
Product Transfer
Return Order
Inventory Count
Inventory Update APIs
Validate
1. Quantity Valid
2. Source valid
3. Other validation Rules
Update Damaged Quantities in child Entity (Batch/SP)
Roll up Changes to Parent Product Item
Create Additional Transaction Logs
Update QtyAvailable using formula
Inputs:
Product Item
QuantityDamaged
Related Entity
Outputs:
Success/Failure
Transaction Log Id
Manual Adjustments
Available Qty = QOH - QtyAllocated - QtyDamaged
Formula Builder/ OOTB default formula
Admin Configures Formula to compute available quantity

[Lucid Link]